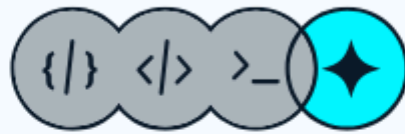


Change Report

Team 8: Ben and Bensons



B&B

CODE. CREATE. CONNECT.

Team Members: Alyssa Skipper, Benjamin Senior, Benson Chow, Chloe Ward, Florian Mengkris, Hannah Thomas, James Ingram, Olivia Spencer

Introduction

Approach to Planning

We created a new Gantt Chart for Assessment 2 to ensure that everything is delivered in a reasonable timeframe. To ensure that every deliverable was getting equal attention, we split the team into subteams that focused on specific tasks. We decided that Alyssa and Chloe would focus mainly on making changes to the written deliverables that we inherited from the previous team, with frequent discussions with the people working on the code so the architecture and risk documentation can be altered quickly and efficiently. We decided that James would focus on updating the code that the previous team had written, while Ben and Florian added new elements of code.

Tracking and Managing Changes

We ensured that every change we made to the original documents were written in an alternate colour. We decided that any edits to the original text that maintained the original intention of the previous group would be written in red and any new ideas or additions would be written in purple. Every change must also be written up in this document immediately so we do not forget what edits were made.

Tools Used

In order to deal with the documents, we used Google Workspace apps. Mainly, we used Google Drive and Google Docs. Google Drive was used to store and organise both the deliverables that we were creating and the deliverables from the previous team that we were editing. Google Docs was used to write and edit these documents. We particularly made use of the ability to change text colours to clarify what changes were made to each document. We also utilised the comment feature in order to interact with each other and point out any errors or uncertainties in a specific yet informal way. On occasion, we also made use of Google Sheets and Google Forms.

We used GitHub for version control of the code. GitHub allowed us to track changes made to the code as there is documentation of every push made to the code. GitHub also has branching functionality which allows multiple members of the team to make edits to the code while ensuring that there are no conflicts with what has been written. The branches can also be merged back into main when the code is complete.

Despite the previous team using IntelliJ, we decided to continue using Visual Studio Code for implementation as we were already familiar with that IDE since we had used VSCode for assessment 1. VSCode also is integrated with GitHub so it allows pushes to be made easily. We did utilise IntelliJ for testing as there is compatibility with JUnit automated testing, which we relied heavily on for unit and integration testing

Changes to deliverables

Some changes had to be made to more than one of the deliverables. An example of this is the formatting changes to the documents. None of the inherited deliverables included references or bibliographies so it was difficult to discover where the information they had cited had originated from. The ideas likely came from either the lectures or from [1], but this cannot be verified so we decided to rewrite the affected text to rely on project-specific evidence.

The original and updated versions of the documents can be found on our [website](#).

Assessment 2 required implementation of the full product brief (expanded events leaderboard and achievements), which increased system complexity and motivated clearer separation of concerns and stronger architectural documentation. This required both documentation updates and implementation changes, increasing the need for rigorous change tracking.

Requirements

Overview of Changes

Following the initial requirements specification, the requirements document was reviewed to ensure clarity, consistency and traceability in light of the refined understanding of the system gained after further team discussion and alignment with the project scope. No changes were made to the fundamental purpose or scope of the system. Instead, changes focused on improving the quality of the specification by removing ambiguity, improving consistency and ensuring alignment between user, functional and non-functional requirements.

Addition of System Summary of Needs (SSoN)

A System Summary of Needs was added to the requirements document, as this had not been explicitly included in the original version. The purpose of introducing the SSoN was to provide a concise, high-level description of the system that clearly communicates its overall goal, target users and key constraints to all stakeholders.

The SSoN does not introduce any new requirements or alter the scope of the system. Instead, it summarises the agreed intent of the project based on the existing user requirements and the original project brief, providing clear context for the detailed requirements that follow.

User Requirement Changes

Several targeted changes were made to the user requirements to improve clarity, completeness and consistency while maintaining the original intent and scope of the project.

Refinement of Existing User Requirements

The description of UR_DISPLAY was revised to improve clarity and precision. The original wording was vague and repetitive, whereas the updated description more clearly distinguishes between responsiveness to different screen sizes and accessibility support for users with visual impairments.

The priority of UR_SYSTEM was changed from should to shall. This change reflects the importance of cross-platform compatibility and OS independence as a core constraint of the system, rather than an optional or desirable feature. The functionality itself was not changed; only its priority was clarified to better reflect the stakeholders' expectations.

Additional clarification was added to UR_TIME_LIMIT to explicitly state that the five-minute timer only counts down while the game is unpaused. This clarification was introduced to remove ambiguity and ensure consistent interpretation between development and testing, without changing the agreed duration of gameplay.

Refinement of Game Completion Requirements

The original user requirements relating to game completion (UR_GAME_COMPLETION) was split into three distinct use requirements: UR_WIN, UR_LOSE and UR_END. This change was made to reduce ambiguity and improve clarity by separating the different aspects of game completion into clearly defined outcomes.

Specifically, UR_WIN defines the condition under which the player successfully completes the game, UR_LOSE defines the failure condition when the time limit expires and UR_END specifies the system behaviour that occurs once the game has concluded. Separating these concerns improves traceability as each requirement can now be independently referenced, implemented and tested.

This change does not alter the behaviour or scope of the system; it formalises behaviour that was previously grouped under a single requirement and ensures that success conditions, failure conditions and post-completion behaviour are explicitly specified. The subsequent functional and non-functional requirements that referenced UR_GAME_COMPLETION were updated to reference the new corrected user requirements.

Addition of New User Requirements

A number of new user requirements were added to address gaps identified during review of the original specification. These requirements do not introduce new scope but instead make previously implicit expectations explicit.

Several requirements were added to clearly define core game constraints and outcomes including single-player gameplay (UR_SINGLE) and age-appropriate content (UR_FAMILY). These requirements formalise assumptions that were already present in the project brief and discussions but had not been explicitly captured.

Additional requirements were introduced to improve usability and accessibility, including audio support and muting (UR_AUDIO, UR_VOLUME), clear instructions accessible from the main menu (UR_INSTRUCTIONS) and appropriate difficulty for inexperienced players (UR_EASE). These additions improve the user experience while remaining consistent with the original system goals.

Optional features such as saving and loading (UR_SAVE) and character customisation (UR_CHAR) were explicitly marked as may requirements. This ensures they are clearly distinguished from mandatory functionality and avoids misinterpretation during implementation.

Finally, several requirements were added to formalise gameplay structure and quality, including the presence of hidden events, hindering events and beneficial events (UR_HIDDEN, UR_HINDERANCE, UR_BENEFIT) as well as expectations around system efficiency (UR_EFFICIENT) and documentation quality (UR_DOC). These requirements improve completeness and maintainability without expanding the system's functionality.

Functional Requirements Changes

A small number of targeted changes were made to the functional requirements to improve completeness, consistency with user requirement priorities and remove duplication.

Addition of New Functional Requirements

Two functional requirements were added to explicitly support existing user requirements.

FR_MUSIC and FR_VOLUME_ADJUST were introduced to formally specify that the player can mute/unmute audio and adjust volume via the pause or settings menu. These requirements support UR_AUDIO and UR_VOLUME and ensure that all audio-related user expectations are fully implemented at the functional level.

FR_INSTRUCTIONS_MENU was added to explicitly support UR_INSTRUCTIONS, which previously has no corresponding functional requirement. This addition improves traceability by ensuring that the requirement for clear, accessible instructions is implemented at the functional level. The requirement does not introduce new functionality but formalises behaviour that was already expected within the system.

Similarly, FR_SAVE_LOAD was added to explicitly support UR_SAVE providing functionality for saving and loading game progress. This requirement clarifies behaviour that was previously implicit and formalises an optional feature for implementation.

FR_CHARACTER_SELECTION was added to support UR_CHAR allowing players to select from multiple character appearances before starting the game. This ensures traceability for optional customisation features without introducing new system scope.

The functional requirement FR_WIN_LOSE_SCREEN was replaced with FR_END_SCREEN to remove duplication and improve clarity within the functional specification. Both requirements described the same system behaviour, namely the display of an end-of-game screen following either a win or a loss. This replacement consolidates this behaviour into a single, unambiguous requirement ensuring a single source of truth for end-of-game functionality. The revised requirement also aligns more clearly with the newly introduced user requirement UR_END, improving traceability between user and functional requirements.

Alignment of Functional Requirement Priorities

Several functional requirements, including FR_CLOCK, FR_ACTIVE_CHALLENGE AND FR_FOOD were updated to use the modal verb shall instead of should. This change was made to ensure consistency with their corresponding user requirements, which were defined as mandatory.

These changes do not alter system behaviour but clarify implementation expectations and improve consistency between user and functional requirements.

Non-Functional Requirement Changes

To fully support UR_EASE regarding difficulty and usability, NFR_EASE was added. This requirement specifies that the game shall be playable without frustration for inexperienced players and should be learnable within the first two minutes of gameplay. This ensures that user expectations regarding ease of play are explicitly captured and traceable, complementing existing NFRs such as NFR_INTUITIVE.

Architecture

Overview of Changes

At the start of Assessment 2, the team inherited an existing implementation and documentation. Before extending functionality to meet the full product brief, a refactoring phase was undertaken to reduce architectural degradation and mitigate progressions towards a “ball of mud” anti-pattern. Due to time constraints, it was not feasible to fully remove the tight coupling; however, key responsibilities were clarified and coordination logic was formalised to improve maintainability and enable feature completion. The revised architecture reflects a more mature understanding of the system structure, clearer separation of responsibilities and the architectural implications of using the libGDX framework.

The component evolution table was fully revised to reflect the implemented architecture, including the replacement of a planned StateManager with the libGDX Screen lifecycle, the distribution of input handling, the introduction of multiple gameplay screens and the formalisation of specialised managers such as BuildingManager, AudioManager and LeaderboardManager.

The overall architectural style remained layered and object-oriented; however, responsibility boundaries were refined and a coordination layer was explicitly formalised to reflect the implemented system.

CRC cards on the website were updated to reflect the final implemented architecture, including the expanded screen set (e.g. UsernameScreen, AchievementsScreen, building interiors such as PiazzaScreen) and item specialisation (Collectable -> EnergyDrink, Torch).

Expansion of Architectural Views

During revision, the architectural documentation was extended to include additional UML types, specifically package, component and state diagrams.

These additions were introduced to complement the existing class and sequence diagrams and to provide architectural perspectives at multiple abstraction levels. Package diagrams were added to document code organisation, component diagrams to describe major subsystems and their dependencies and state diagrams to model full gameplay flow across menus, gameplay and buildings and end states

This change improves architectural completeness and supports clearer communication of both static structure and system-level behaviour.

Formalisation of a Coordination Layer

The architectural style description was revised to explicitly introduce a manager/coordination layer. This layer includes specialised components such as BuildingManager, ItemManager and LeaderboardManager.

This change was made to clarify where cross-cutting gameplay responsibilities are handled including building transitions, item interactions and score persistence. Formalising this layer prevents presentation screens and domain entities from becoming overly coupled or overloaded with coordination logic.

This improves separation of concerns, increases cohesion with individual layers and provides a clearer architectural structure for future extensions.

Refinement of Layer Responsibility

The responsibilities of the presentation, domain, infrastructure and coordination layers were rewritten to more precisely reflect the implemented system. The revised descriptions clarify that screens are responsible for rendering and local input handling, domain classes encapsulate gameplay state and rules, managers coordinate multi-entity behaviour and the infrastructure layer controls lifecycle and framework integration.

This refinement was introduced to reduce ambiguity present in the earlier documentation and to ensure that the architectural descriptions align with actual code responsibilities.

Behavioural Architecture Clarification

The behavioural architecture section was significantly expanded. The revised version explicitly distinguishes external actors from internal system objects within sequence modelling and states that all sequence messages correspond to implemented class operations.

These changes were introduced to improve correctness, consistency and traceability between behavioural diagrams and structural models and to ensure that the documented runtime behaviour reflects actual system execution. This directly addresses earlier feedback by separating the external actor from the Player object and ensuring that each message corresponds to an operation in the receiving class.

Extension of Architectural Evolution Documentation

The iteration section was extended to include an additional architectural stage documenting refinements made during final implementation. This includes the formalisation of specialised manager classes and evidence of architectural stability as new features were integrated.

This change was introduced to more accurately document architectural progression and to demonstrate that the final system accommodated additional functionality without requiring structural redesign.

CRC-Driven Responsibility Clarification

The CRC iteration section was rewritten to emphasise its role in driving architectural refinement. Responsibilities were clarified, unnecessary centralised controllers were removed and alignment with the libGDX screen framework was documented. This improves traceability between requirements, architectural decisions and implemented components and formally documents the rationale behind responsibility allocation.

The CRC-to-requirements mapping page on the website was updated to reflect the final implemented architecture. This included adding newly introduced screens, collectable specialisation and building-specific gameplay contexts to ensure accurate traceability between requirements and CRC components. It was also updated to reflect revised Req2 IDs and newly introduced requirements, maintaining traceability between requirements and CRC constructs.

Strengthening of Architectural Justification

Both abstract and concrete architecture justifications were rewritten to move beyond theoretical design intent and instead reflect architectural outcomes observed during implementation.

The revised justifications now explicitly address scalability, maintainability, framework-driven trade-offs and architectural stability providing a more rigorous evaluation of the final architecture.

Method Selection and Planning

We used the same software development methodology as the previous team (Scrum) so there was very little to change in that regard. Like the previous team, we also meet twice a week in person and implement weekly sprints. However, unlike the previous team we did not use Kanban boards, so any mentions of them were removed.

We also used many of the same tools as the previous team: Google Drive, WhatsApp, LibGDX, IntelliJ, and GitHub. However, we also used some additional tools. We also used Discord to communicate in addition to WhatsApp as the channels allowed us to differentiate what each message was about and ensure that everything was easy to find. Furthermore, unlike the previous team, we mostly used VSCode to create the code for the game as our team members were familiar with this IDE. We did still use IntelliJ but we mostly used this IDE for testing purposes as the extensions (such as the JUnit extension) were seen as more useful.

The previous team assigned members to specific deliverables to ensure that each part of the project was completed, however the team members were not restricted to these roles. This did not seem reasonable to us as we thought this approach increased the likelihood that parts of the project would be neglected or that some members of the team will do less work than others. We also replaced mentions of 'daily' communication with 'frequent' communication as we deemed it unnecessary for our group to speak daily.

Various updates to the diagrams, found on the [website](#), were made. Most of the diagrams and tables that the original team created remain on the website with the exception of their original work package table as that was fully reformatted.

Firstly, we added more Gantt charts to give us a framework for our new deadlines and deliverables. As well as tasks, we also specified key dates that may pose a challenge for our group, such as the end of term (meaning we will no longer be able to meet in person), the official hand-in date, and the presentation date. Despite creating brand new charts, we kept the same colour scheme and formatting of the previous team to keep everything cohesive.

Secondly, we created new Work Breakdown Structures (WBS). Similarly to the Gantt chart, this was also made with cohesion in mind but was updated to reflect the different brief for Assessment 2.

Thirdly, we added to the deliverable table. This was virtually unchanged except for the additions of the new deliverables that are expected for Assessment 2. This table reflects both the official deadline (January 13th) and our internal deadline (December 31st).

Finally, we changed the work package table. We found this table difficult to understand at first so we decided to completely overhaul the formatting. We decided that it was very hard at first to determine which work package each task belonged to- which we deemed was the most important element of this table. As a result, we decided to add a new column which clearly stated which work package a task belonged to. We also colour coded each row based on the work package and changed the numbering system. Previously the second task of WP1, for example, would be numbered as 2.1 and we changed it to be 1.2 as this system puts the work package first. These large changes and the fact that we reformatted the original table led to us removing the original table from the website, but the information that it conveyed is still available in our new table.

We also added five more milestones to describe the new tasks that were required for assessment 2. These are shown below:

M6: Reorganise the team - Due date: Wednesday 26th November

M7: Complete introductions for all deliverables - Due date: Wednesday 10th December

M8: Complete all deliverables for internal checks and tests - Due date: Wednesday 31st December

M9: Project completion and submission - Due date: Tuesday 13th January

M10: Presentation - Due date: Friday 16th January

We also updated the week-by-week Gantt chart write up. Firstly, we made sure that it was obvious which parts of the project evolution was created by the original team. We did not make any changes to what the previous team had written as we found it important to maintain their reasoning for making the decisions they did in the initial game development. We also added images of the first and final Gantt charts as we thought the images helped readers of the document as they did not have to leave and go to the website to view the charts. We added our own weekly update to this document.

Risk Assessment

The description of the risk management was slightly altered from the previous group. Parts of paragraph 1 were rewritten for clarity. Paragraph 2 was rewritten to remove all mentions of the second table that we decided to remove. We decided to remove the *Management risk register* table from the Risk Assessment document as the contents of the table were specific to the previous group and were not applicable to us; most of the risks identified suggested

various disagreements within the group structure, an issue we have not encountered. Paragraph 3 also had mentions of this second table removed, as well as changes to better reflect our team structure. There were minimal changes to the final paragraph- just changing the name of the main risk manager to be a member of our team. Most of this paragraph was maintained as our team also decided to utilise the same monitoring and reviewing system as the previous team.

There was also a lack of citations from the previous group. We attempted to discover where the ideas and information had come from but we were unable to find a source that used the exact same type system or suggestions of where they had found the standard risk register table.

Nine new risks were added to the risk register (R20-28 inclusive) that the team encountered as we engaged with the inherited code and documentation from the previous group. There were two main elements that many of the risks pertained to; familiarising ourselves with and editing the previously created code and documentation, and the high possibility of a breakdown in communication and productivity over the Christmas period. The other risks were specific to Assessment 2, being about learning how to create tests and finding individuals to complete our user evaluation forms.

Updates to the Risk Register:

ID	Addition, Large Edit or Small Edit (A/LE/SE)	Description
All	SE	Changed the risk owner from members of the previous team to members of our team.
R1, R6, R8	SE	Removal of mentions of Kanban boards as our team did not use these as we prefer to use a fully SCRUM methodology.
R3	SE	Changed the number of marks to better reflect number of team members
R5	LE	Much of the original risk was about handing over code. This risk was not deleted as the mitigation technique of maintaining readable code was deemed useful as code still must be easily followed by multiple team members, as well as the examiner.
R8	SE	Changes to phrasing to make the risk more general.
R8	SE	Changed the likelihood and severity. They were previously medium and low, and are now low and medium. This switch was made as we do not believe that it is likely for a member of the team to leave the project but it would impact the project negatively if it were to occur.
R9	SE	Changed the likelihood rating from medium to low as we have had minimal disagreements over the course of the project.
R11, R12	SE	Edits to the description to ensure it is readable.
R14	LE	Removal of mentions of handing the project over to another team as that was a requirement of Assessment 1 and will not occur after Assessment 2.
R14	SE	Changed the likelihood rating from medium to low as the client is unlikely to change the requirements for the project during the course of Assessment 2.

References

[1]I. Sommerville, Software engineering, 10th ed. Boston, Mass. Amsterdam Cape Town Pearson Education Limited, 2015.

Available:

<https://dn790001.ca.archive.org/0/items/bme-vik-konyvek/Software%20Engineering%20-%20Ian%20Sommerville.pdf>